

Tecnatura Superior en Desarrollo de Software

Anexo 04 — Inyección de Dependencias e Inversión de Control

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué es la Inversión de Control y la Inyección de Dependencias?

Cuando una clase crea internamente los objetos que necesita, controla su propio ciclo de vida y el de sus colaboradores. Ese control puede invertirse: en lugar de que la clase decida qué implementación usar, alguien externo se la proporciona ya construida. Ese principio se llama **Inversión de Control** (IoC, *Inversion of Control*).

La **Inyección de Dependencias** (DI, *Dependency Injection*) es la técnica más común para aplicar IoC: las dependencias de una clase se declaran como parámetros de su constructor y el *contenedor* de DI las proporciona automáticamente al instanciar la clase. El código de la clase nunca escribe `new` para sus colaboradores — solo declara qué necesita.

Principio DIP (Dependency Inversion Principle — SOLID): los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones. La inyección de dependencias es el mecanismo que hace ese principio operativo en tiempo de ejecución.

Analogía del mundo real

Imaginá un restaurant. El cocinero necesita ingredientes para preparar un plato. Si el cocinero saliera a comprar los ingredientes cada vez que cocina, dependería de dónde conseguirlos, a qué precio y en qué condición. En cambio, el restaurant

tiene un **almacén centralizado**: alguien deja los ingredientes preparados al alcance del cocinero antes de que empiece a cocinar. El cocinero solo declara qué necesita — el almacén lo provee.

En el código, el **contenedor de dependencias** es ese almacén. Las clases declaran sus necesidades en el constructor. El contenedor las construye y entrega, resolviendo las dependencias de forma automática, en cascada y con el ciclo de vida correcto.

El problema que resuelve: acoplamiento oculto

El problema del acoplamiento fuerte aparece cuando una clase instancia sus propias dependencias. Veamos un ejemplo con un caso de uso de registro de pedidos (`PlaceOrderUseCase`):

SIN INYECCION DE DEPENDENCIAS

El caso de uso crea su propio repositorio con `new`. Está acoplado a la implementación concreta — no se puede reemplazar por un doble de prueba ni por una implementación distinta.

```
1 public sealed class PlaceOrderUseCase
2 {
3     // MAL: crea la dependencia internamente
4     private readonly SqlOrderRepository _repo
5         = new SqlOrderRepository();
6
7     // el UseCase conoce la clase concreta - viola DIP
8     // imposible de testear sin base de datos real
9     public async Task ExecuteAsync(PlaceOrderRequest r)
10         => await _repo.AddAsync(new Order(r.ProductId));
11 }
```

CON INYECCION DE DEPENDENCIAS

El caso de uso declara que necesita un `IOrderRepository`. No sabe — ni le importa — cuál es la implementación concreta. El contenedor la provee al instanciar la clase.

```
1 public sealed class PlaceOrderUseCase
2 {
3     // BIEN: depende de la abstracción – viola nada
4     private readonly IOrderRepository _repo;
5
6     // el contenedor inyecta la implementación concreta aquí
7     public PlaceOrderUseCase(IOrderRepository repo)
8         => _repo = repo;
9
10    public async Task ExecuteAsync(PlaceOrderRequest r)
11        => await _repo.AddAsync(new Order(r.ProductId));
12 }
```

El contenedor de .NET: `IServiceCollection`

.NET incluye un contenedor de dependencias integrado. Su punto de entrada es la interfaz `IServiceCollection`, que actúa como un **catálogo de registros**: cada entrada asocia una abstracción (interfaz) con una implementación concreta (clase) y define su *ciclo de vida*.

Al iniciar la aplicación, `Program.cs` construye ese catálogo. A partir de ese momento, cada vez que el framework necesita crear un objeto que depende de una interfaz registrada, consulta el catálogo y resuelve la cadena completa de dependencias de forma automática.

Flujo de resolución:

`Program.cs` registra → contenedor construye el catálogo → request llega → contenedor resuelve `IPlaceOrderInputPort` → instancia `PlaceOrderUseCase` → inyecta `IOrderRepository` → instancia `SqlOrderRepository` → inyecta `OrderDbContext` → todo resuelto en cascada.

Métodos de registro: `Add___<TInterface, TImplementation>()`



Regla de compatibilidad de ciclos de vida: un servicio solo puede recibir dependencias de ciclo de vida *igual o mayor* al propio. Un *Singleton* no puede recibir un *Scoped* — el *Scoped* quedaría retenido más allá de su request. Un *Scoped* no puede recibir un *Transient* sin riesgo de inconsistencias. El compilador no detecta estos errores — el contenedor de .NET lanza una excepción en tiempo de ejecución si la validación está habilitada.

Organización por capas: métodos de extensión

Cuando una aplicación crece, registrar todos los servicios directamente en `Program.cs` produce un archivo enorme y difícil de mantener. La convención de Clean Architecture es que **cada capa expone un método de extensión propio** que registra únicamente sus implementaciones. `Program.cs` solo encadena esos métodos.

Un **método de extensión** en C# es un método estático que actúa como si fuera un método de instancia de otro tipo, gracias al modificador `this` en el primer parámetro. Al extender `IServiceCollection`, el método aparece directamente sobre `builder.Services` sin necesidad de instanciar ninguna clase auxiliar.

```

1 // Ejemplo neutro: capa Application de un dominio de pedidos
2 // El namespace del archivo es el del paquete – truco para evitar using extra
3 namespace Microsoft.Extensions.DependencyInjection;
4
5 using Orders.Application.Ports;
6 using Orders.Application.UseCases;
```

```
7
8 // clase estática – nunca se instancia
9 public static class DependencyContainer
10 {
11     // "this IServiceCollection" → convierte este método estático en extensión
12     // retorna IServiceCollection → permite el encadenamiento fluente
13     public static IServiceCollection AddOrderUseCases(
14         this IServiceCollection services)
15     {
16         // interfaz → implementación, ciclo de vida Scoped
17         services.AddScoped<IPlaceOrderInputPort, PlaceOrderUseCase>();
18
19         // retornar services permite encadenar más llamadas en Program.cs
20         return services;
21     }
22 }
```

El truco del namespace

Al declarar `namespace Microsoft.Extensions.DependencyInjection`, el método de extensión vive en el mismo namespace que `IServiceCollection`. Esto significa que cualquier archivo que ya tenga acceso a ese namespace — como `Program.cs`, donde el compilador lo importa implícitamente al agregar el paquete — puede usar el método sin escribir ningún `using` adicional.

Por qué funciona: C# busca los métodos de extensión en todos los namespaces que están en el alcance del archivo donde se los invoca. `Program.cs` en un proyecto ASP.NET Core ya tiene `Microsoft.Extensions.DependencyInjection` en su alcance global porque el SDK lo importa automáticamente. Por lo tanto, el método `AddOrderUseCases()` es visible sin un `using` extra.

Resultado en `Program.cs`

```
1 // Program.cs – sin ningún using adicional de las capas internas
2 // cada capa registra solo sus propias implementaciones
3 builder.Services
4     .AddOrderUseCases() // Orders.Application
5     .AddOrderInfrastructure() // Orders.Infrastructure
6     .AddOrderPresenter(); // Orders.InterfaceAdapter
```

Marco teórico complementario

¿Por qué no usar el Service Locator?

El patrón *Service Locator* consiste en pedir las dependencias al contenedor desde dentro de la clase, en lugar de recibirlas por constructor. Aunque ambos enfoques usan el mismo contenedor, el Service Locator oculta las dependencias reales de la clase: la firma del constructor no refleja lo que la clase necesita, lo que dificulta las pruebas unitarias y el análisis estático del código.

ERROR COMUN — SERVICE LOCATOR

La clase pide sus dependencias al contenedor desde adentro. Las dependencias son invisibles desde afuera.

```
1  public sealed class PlaceOrderUseCase
2  {
3      private readonly IServiceProvider _sp;
4      public PlaceOrderUseCase(IServiceProvider sp)
5          => _sp = sp;
6      public async Task ExecuteAsync(PlaceOrderRequest r)
7      {
8          // MAL: la clase busca su propia dependencia
9          var repo = _sp.GetRequiredService<IOrderRepository>();
10         await repo.AddAsync(new Order(r.ProductId));
11     }
12 }
```

BUENA PRACTICA — DI POR CONSTRUCTOR

Las dependencias son visibles en el constructor. Cualquier test puede pasar un doble sin configurar el contenedor.

```
1 public sealed class PlaceOrderUseCase
2 {
3     private readonly IOrderRepository _repo;
4     // BIEN: dependencia explícita y reemplazable
5     public PlaceOrderUseCase(IOrderRepository repo)
6     => _repo = repo;
7     public async Task ExecuteAsync(PlaceOrderRequest r)
8     => await _repo.AddAsync(new Order(r.ProductId));
9 }
```

Cómo el contenedor resuelve la cadena de dependencias

Cuando `Program.cs` llama a `AddOrderUseCases()`, no crea ningún objeto todavía — solo registra la asociación entre interfaces y clases. La creación ocurre cuando llega el primer request que necesita un `IPlaceOrderInputPort`. En ese momento el contenedor:

1. Busca en el catálogo la implementación de `IPlaceOrderInputPort` → encuentra `PlaceOrderUseCase`.
2. Inspecciona el constructor de `PlaceOrderUseCase` → necesita `IOrderRepository`.
3. Busca la implementación de `IOrderRepository` → encuentra `SqlOrderRepository` y `y`.
4. Inspecciona el constructor de `SqlOrderRepository` → necesita `OrderDbContext`.
5. Crea `OrderDbContext`, luego `SqlOrderRepository`, luego `PlaceOrderUseCase`.
6. Entrega la instancia completamente construida al punto de inyección.

Condición necesaria: cada clase que el contenedor debe instanciar debe tener un constructor cuyas dependencias estén todas registradas en el catálogo. Si falta algún registro, el contenedor lanza una excepción en tiempo de ejecución con el mensaje: *"Unable to resolve service for type X while attempting to activate Y."*

Errores comunes

1. Registrar la implementación concreta en lugar de la interfaz

ERROR COMUN

Registrar la implementación concreta como servicio — el llamador recibe la clase concreta y el desacoplamiento desaparece.

```
1 // MAL: registra la clase concreta como servicio
2 services.AddScoped<PlaceOrderUseCase>();
3 // el llamador debe pedir PlaceOrderUseCase — no IPlaceOrderInputPort
```

BUENA PRACTICA

Registrar siempre la interfaz como clave — la implementación concreta es el detalle que el contenedor conoce, no el llamador.

```
1 // BIEN: interfaz → implementación
2 services.AddScoped<IPlaceOrderInputPort, PlaceOrderUseCase>();
```

2. Inyectar un Scoped dentro de un Singleton (Captive Dependency)

ERROR COMUN — CAPTIVE DEPENDENCY

Un Singleton que recibe un Scoped retiene esa instancia para siempre, corrompiendo los datos de requests futuros.

```
1 // MAL: OrderCache es Singleton y recibe un Scoped
2 services.AddSingleton<IOrderCache, OrderCache>();
3 services.AddScoped<IOrderRepository, SqlOrderRepository>();
4 // OrderCache(IOrderRepository) → SqlOrderRepository queda retenido
5 // todos los requests futuros compartirán el mismo DbContext — datos corruptos
```

BUENA PRACTICA

Un Singleton solo puede recibir dependencias Singleton. Si necesita un Scoped, usar `IServiceScopeFactory` para crear un scope propio.

```
1 // BIEN: Singleton recibe Singleton
2 services.AddSingleton<IOrderCache, OrderCache>();
3 services.AddSingleton<ISystemClock, SystemClock>();
4 // OrderCache(ISystemClock) → ciclos de vida compatibles
```

3. Olvidar registrar una dependencia**ERROR COMUN**

El UseCase está registrado pero su repositorio no lo está. El contenedor lanza una excepción al primer request.

```
1 // MAL: el UseCase depende de IOrderRepository – no está registrado
2 services.AddScoped<IPlaceOrderInputPort, PlaceOrderUseCase>();
3 // falta: services.AddScoped<IOrderRepository, SqlOrderRepository>();
4 // resultado en runtime: InvalidOperationException
```

BUENA PRACTICA

Cada capa registra *todas* sus implementaciones en su propio `DependencyContainer`.

```
1 // BIEN: registro completo por capas
2 builder.Services
3     .AddOrderUseCases() // Application – registra UseCases
4     .AddOrderInfrastructure() // Infrastructure – registra repositorios
5     .AddOrderPresenter(); // InterfaceAdapter – registra presentadores
```

Qué NO hacer

QUE NO HACER

Registrar todos los servicios directamente en `Program.cs` sin organización por capas.

```
1 // MAL: Program.cs conoce todas las implementaciones concretas
2 builder.Services.AddScoped<IPlaceOrderInputPort, PlaceOrderUseCase>();
3 builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();
4 builder.Services.AddScoped<IOrderPresenter, OrderViewModel>();
5 // cuando hay 50 UseCases, Program.cs tiene 150 líneas de registros
```

BUENA PRACTICA

`Program.cs` solo encadena los métodos de extensión de cada capa. Cada capa se auto-registra y es responsable de sus propios detalles de implementación.

```
1 // BIEN: Program.cs no conoce ninguna implementación concreta
2 builder.Services
3     .AddOrderUseCases()
4     .AddOrderInfrastructure()
5     .AddOrderPresenter();
```